Contents

music-room — User Manual **1**
1. What this repository is . . . 1
2. Directory map . . . 1
3. The retrieval path . . . 2
4. Daily workflows . . . 2
5. The document layer . . . 4
6. The standards library . . . 5
7. Using Obsidian . . . 5
8. Maintenance rules . . . 5
9. Configuration and secrets . . . 6
10. Onboarding a teammate . . . 6
11. Troubleshooting . . . 6
12. Glossary . . . 7
Related . . . 8

music-room — User Manual

How to work in this repository: where things live, who updates what, and what never to touch.

Read this once. After that, [[HOME]] is your daily entry point.

---

1. What this repository is

An École 42 group project: a collaborative music platform with live track voting, real-time playlist editing, and delegated music control. Current work scope is the **backend**.

The repository carries four layers. They are **views over the same files**, never copies.

Layer	Lives in	Audience	Question it answers
AI retrieval	`brain/`	agents	"what is the context, cheaply?"
Human navigation	Obsidian vault at the repo root	you	"how does this connect?"
Design record	`docs/SDD.md`	evaluators, new joiners	"how does the whole thing hang together?"
Security conformance	`docs/owasp/`	everyone writing code	"what does OWASP-conformant mean here?"
Authoritative docs	`docs/technical-guide/`	everyone	"what rule must my code follow?"
Implementation	`MusicRoom/`	compiler	"what does the code actually do?"

---

2. Directory map

```
music-room/
  HOME.md                human entry point
  AGENTS.md              shared contract for ANY AI agent
  CLAUDE.md              Claude Code specifics; points to AGENTS.md
```

```
GEMINI.md                Gemini CLI specifics; points to AGENTS.md
.agent/                  unattended run driver (run.sh, STOP, git-ignored runs/)
.env.example             every config key, placeholder values
MusicRoom/               backend source code (.NET 10)
MusicRoomApp/            React Native client - same repo, separate root
brain/                   knowledge layer (small on purpose)
    MEMORY.md            the router
    project-registry.md  what components exist
    projects/            one card per component
    constraints/         the 42 subject, distilled
    architecture/        how the system is shaped
    decisions/           ADRs
    migrations/          schema transition log
    lessons/             durable gotchas
    standards-router.md  pointer into the standards library
    knowledge-map.md     how every note relates
docs/
    subject/             the 42 subject PDF (authoritative, binary)
    technical-guide/     19 engineering standards, .md + .pdf
    prompt-library/      reusable agent prompts (EN canonical + TR reading aid)
    SDD.md               consolidated software design document
    DELIVERY-PLAN.md     phase order, decision queue, vertical slices
    owasp/               conformance target, checklists, threat register, vendored ASVS
    api/                 ENDPOINTS.md contract, and openapi.json once code exists
    specs/               disposable work orders, with the self-approval gate
    USER-MANUAL.md       this file
    obsidian-setup.md    vault configuration
    build-pdf.sh         regenerates every PDF
reference/               READ ONLY - study material, never edited
```

The two path rules

1. **Stay in your root.** Backend code under `MusicRoom/`, client code under `MusicRoomApp/`. Neither writes into the other; the contract between them is `docs/api/ENDPOINTS.md`.
2. **`reference/` is read-only.** Open it to learn a pattern. Then write your own code under `MusicRoom/`. Never edit it, never build it, never `ProjectReference` it, never copy a file out of it. It is net6 with stale packages, and subject IV.1 forbids committing third-party source into the repository.

---

3. The retrieval path

`MEMORY -> project-registry -> project card -> architecture / ADR / standard -> source code`

Each arrow narrows deliberately. An agent that follows it reads four small files instead of the whole workspace. A human who follows it gets oriented in about five minutes.

Never skip a step by searching the whole vault. That is the habit this structure exists to remove.

---

4. Daily workflows

Running an agent

Every agent reads `AGENTS.md`: retrieval path, write targets, authority order, non-negotiables, definition of done, and when to escalate. Claude Code and Gemini CLI have thin per-tool files that point back to it.

For unattended runs, the rule is narrow on purpose: **an agent implements specs that sit inside decisions already made.** It never chooses architecture, adds a dependency, or touches authentication without a human — subject section II requires the team to own and defend its decisions. The seven-question gate in [[specs]] makes that enforceable, and [[agent-runner]] adds a cycle cap and a `STOP` file on top.

Finding the next piece of work

Open [[delivery-plan]]. It is ordered by dependency: Phase 0 verify, Phase 1 scope, Phase 2 close the blocking decisions, Phase 3 fill the design gaps, Phase 4 build in vertical slices.

Status is **not** tracked in the plan. It lives in [[sdd]] §16, so there is one place to look and one place to lie about. Open decisions live in [[sdd]] §17.

Starting a work session

1. Open [[HOME]].
2. Check *Current state* for open decisions.
3. Read the project card for what you are touching — [[musicroom-backend]].
4. Open the standard for the area — via [[technical-guide-index]].

With an agent, paste prompt §1 from [[prompt-library]] instead. It does the same in one turn.

Implementing a feature

1. Read the card and the architecture note.
2. Read the applicable standards. They are the review criteria, so read them before, not after.
3. Write code under `MusicRoom/`.
4. Write the tests the `testing` standard requires for that layer — including the concurrency case if you touched a hotspot.
5. Walk [[owasp-top10-checklist]] and [[owasp-api-top10-checklist]]. Evidence per row, not opinion. Anything you do not fix goes to [[threat-register]] before the PR merges.
6. Open a PR per the `git-workflow` standard.

Making a decision

Decide it yourself first. Then record it:

1. New file `brain/decisions/ADR-XXXX-<slug>.md` — Context, Decision, Consequences, Alternatives rejected.
2. Row in [[decisions-index]].
3. Update any card, architecture note or standard the decision invalidates.
4. Bump the touched standard's version and regenerate its PDF.

Never edit an accepted ADR. Supersede it with a new one. The history of a decision is part of the defence.

Making a design change

A design change is bigger than one commit and smaller than a new project: a new bounded context, a changed interaction flow, a new external interface, a different concurrency mechanism.

1. If it rests on a technology or approach choice, write the ADR first (see above).
2. Update the affected viewpoint section of [[sdd]] — §4 context, §5 composition, §6 logical, §8 information, §9 interface, §10 interaction, §12 concurrency, §13 security, §14 deployment.
3. Update the traceability table in [[sdd]] §16 if the change affects a mandatory requirement.
4. Close or open the matching row in [[sdd]] §17.
5. Regenerate the PDF.

Two rules keep the SDD trustworthy:

- **Never restate ADR rationale in the SDD.** Reference the ADR. Rationale has one home.
- **Never write "implemented" in §16 for something with no code.** The valid statuses are `designed`, `partial`, `not started`, `done`. An inflated table fails at defence.

Prompt §10b in [[prompt-library]] does this pass with an agent.

Reviewing a pull request

1. [[git-workflow]] — process.
2. The standard for the area changed.
3. [[testing]] — what must be proven.
4. For anything touching votes, playlist order or device control: the concurrency prompt (§4).

Preparing the defence

1. [[subject-requirements]] — the checklist you are graded against.
2. [[sdd]] §16 — the traceability table: every mandatory requirement mapped to the design element that satisfies it. §17 lists what is still open, so nothing surprises you.
3. [[decisions-index]] — every choice with its rejected alternatives. Evaluators ask "why not X".
4. [[load-testing]] — the capacity number and the machine it was measured on.
5. [[authentication-authorization]], [[application-security]] — the threat story, including the risks you accepted and wrote down.

---

5. The document layer

Three kinds of document, three different jobs. Do not blur them.

Document	Answers	Changes when
[[sdd]]	"how is the system designed, and is requirement X covered?"	the design changes
[[delivery-plan]]	"what do we build next, and in what order?"	scope or sequencing changes
[[owasp-index]]	"what does secure mean, and what did we accept?"	the conformance scope or a residual risk changes
[[prompt-library]]	"how do I ask an agent for this, and what may it do alone?"	a workflow or a guardrail changes
[[technical-guide-index]] (19 standards)	"what rule must my code follow?"	a rule changes
[[user-manual]] (this file)	"how do I work in this repository?"	a process changes

A rule belongs in a standard, never in the SDD. A design decision belongs in an ADR and is consolidated in the SDD, never duplicated into a standard.

The Software Design Document

[[sdd]] is organised by IEEE 1016 viewpoints. The two sections you will use most:

- **§16 Requirements traceability** — every mandatory subject requirement mapped to the design element that satisfies it, with an honest status. This is the table to walk through at defence.
- **§17 Open design issues** — O1 through O10, each with what it blocks. This is your decision queue.

The SDD **consolidates**; it is not the source of truth for anything. Authority order is printed in its §1.4.

6. The standards library

`docs/technical-guide/` holds 19 standards in nine categories.

- **Markdown is the editable source of truth. PDF is a distribution artifact.** Never edit a PDF.
- Each standard separates **Official requirements** (subject / RFC / vendor documentation — not negotiable by us) from **Project policy** (our decision, could have been different).
- Each folder is `<topic>/STANDARD.md` + `<topic>/STANDARD.pdf`.
- `manifest.json` is the machine-readable inventory. `README.md` is the human index.
- A topic appears in exactly one folder. Standards cross-reference instead of repeating.

Regenerating PDFs

```
./docs/build-pdf.sh
```

Requires `pandoc` and a PDF engine (`tectonic`, already installed). The script regenerates every `STANDARD.pdf`, plus the SDD, this manual and the prompt library — 22 PDFs in total.

Adding a rule

Put it in the existing standard that fits. A new standard folder is a last resort — a library nobody can hold in their head is a library nobody reads.

---

7. Using Obsidian

Obsidian is the human view. It is **optional**: delete `.obsidian/` and every note still reads correctly in any editor or on GitHub.

- Open the **repository root** as the vault, not `brain/`. The Brain and the standards must be able to link to each other.
- Configuration is `.obsidian/app.json` only: wikilinks, attachment folder, and ignore filters for `reference/`, `docs/subject/`, `bin/`, `obj/`, `node_modules/`.
- No community plugin is installed or needed.
- Standards all share the filename `STANDARD.md`, so each declares an `aliases:` line in its frontmatter. `[[secrets-management]]` resolves through it. **Do not delete that line.**

Full detail: [[obsidian-setup]].

---

8. Maintenance rules

Event	What you update	What you do **not** update
New component	a row in [[project-registry]] + its own card	nothing else
New decision	a new ADR + a row in [[decisions-index]]	never edit an accepted ADR
Design change	the affected viewpoint in [[sdd]], plus §16 and §17	do not copy ADR rationale into it
Requirement newly satisfied	the status cell in [[sdd]] §16	do not write “implemented” without code
Open issue resolved	the row in [[sdd]] §17, and the matching `UNKNOWN` in `brain/`	do not delete the row silently — close it
New rule	the existing standard that fits, version bumped, PDF regenerated	do not restate it in `brain/`

Event	What you update	What you do **not** update
Schema change	a row in [[migrations-index]]	nothing else
Hard-won gotcha	a row in [[lessons-index]]	not a log, not a transcript
Unfixed security finding	a row in [[threat-register]]	never delete a row; close it with evidence
Conformance scope change	[[asvs-conformance]] plus an ADR	do not silently widen a claim
Code change	nothing in `brain/`	the Brain tracks decisions, not commits

What never goes in the Brain

Secrets or `.env` values. Logs, transcripts, debug output. Copied source code. Anything the repository or git history already records. Anything true only for today.

`UNKNOWN - requires targeted inspection`

A deliberate, valid answer. It means: do not guess, inspect on demand, then update the note. Replacing it with a plausible guess is worse than leaving it.

---

9. Configuration and secrets

- Every key is listed in `.env.example` with a placeholder. Copy it to `.env` and fill it in.
- `.env` is git-ignored from the first commit.
- **Subject V.8: publicly stored credentials cause direct project failure.** This is the one rule in the subject with a stated automatic-failure consequence.
- A secret that reaches any branch is compromised. Rotate it at the provider **first**, then rewrite history. Deleting the file in a new commit fixes nothing.
- No client ever holds a database connection string, a Supabase key, a YouTube API key, or an OAuth client secret. A mobile bundle can be unpacked.

Full procedure: [[secrets-management]].

---

10. Onboarding a teammate

Hand them this list:

1. Read [[HOME]], then this manual.
2. Read [[subject-requirements]] — 10 minutes, it is what you are graded on.
3. Read [[sdd]] — the whole design in one pass, ending with §16 and §17 so they know what is covered and what is still open.
4. Read [[decisions-index]] and skim every ADR — this is the project's reasoning.
5. Read [[engineering-baseline]] and [[git-workflow]].
6. Read the standard for whatever they will touch first.
7. `cp .env.example .env`, fill it, run the stack.
8. Open the repo root as an Obsidian vault.

Expect about two hours. That is the whole point of the structure.

---

11. Troubleshooting

Symptom	Cause	Fix
A `[[wikilink]]` does not resolve in Obsidian	the target's `aliases:` line was removed, or the file was renamed	restore the alias, or update the link
Graph view is full of noise	ignore filters were cleared	restore `userIgnoreFilters` in `.obsidian/app.json` per [[obsidian-setup]]
`build-pdf.sh` says no PDF engine	`tectonic` missing	`brew install tectonic`, or leave PDFs pending and edit Markdown only
An agent starts scanning the whole repo	it did not read `CLAUDE.md`	prompt §1 from [[prompt-library]]
The Brain contradicts the code	a decision changed without the Brain being updated	prompt §14, then fix the note
[[sdd]] §16 claims more than is true	a status was inflated	reset it to `designed` / `not started`; only code makes a row `done`
An `UNKNOWN` persists after the decision was made	the ADR was written but not propagated	prompt §10b, then check [[sdd]] §17
`dotnet ef` fails on prepared statements	migrations were run through the pooled connection	use `SUPABASE_DB_DIRECT_URL`

---

12. Glossary

Term	Meaning here
Brain	`brain/` — the AI retrieval layer: facts, decisions, pointers. Never source code.
Project Card	one note per component, holding its identity and open questions
ADR	Architecture Decision Record — one decision, its context, consequences and rejected alternatives
Standard	an enforceable engineering rule set in `docs/technical-guide/`
Official requirement	mandated by the subject or an external specification; not ours to change
Project policy	our decision; could have been decided differently, and must be defensible
SDD	Software Design Document — `docs/SDD.md`, the consolidated design view, IEEE 1016 viewpoints
Viewpoint	one perspective on the design (context, logical, information, interface, concurrency…)
Traceability	the §16 mapping from each mandatory requirement to the design element satisfying it
Retrieval path	the fixed narrowing sequence from MEMORY to source code
Concurrency hotspot	vote casting, playlist reorder, control takeover — the three graded race conditions

---

Related

[[HOME]] · [[MEMORY]] · [[sdd]] · [[delivery-plan]] · [[owasp-index]] · [[knowledge-map]] · [[technical-guide-index]] · [[prompt-library]] · [[obsidian-setup]]